

Library Database

Difficulty: Easy

Expect Time: 15 min

Author: Homework

Please implement a library database system to store information of books, including the title, author, and the editions of each book. Each edition has a quantity of borrowable copies. You will also need to implement the following commands to manage the database and book-lending operations.

Database Operations

The library database supports the following commands:

- **Insert**

To insert a book takes two strings, **title** and **author**, an integer **edition**, and a positive integer **quantity** (the initial number of borrowable copies of that edition) as input, adds the book datum into the database, and outputs the message of **Insert author's title, Edition: edition.;** if it does not exist. Otherwise, output the message **Datum already exist.;**.

- **Delete Edition**

To delete a specific edition takes two strings, **title** and **author**, and an integer **edition** as input, locates and removes the specific edition matching the **title** and **author**, and outputs the message **Delete author's title, Edition: edition.;** if it exists. Otherwise, output the message **Datum doesn't exist.;**. **If any copy of the specified edition is currently borrowed and not yet returned**, the deletion must be refused and the message **Outstanding borrows exist.;** must be output instead.

- **Find Book**

To find data takes two strings, **title** and **author** as input, finds the book datum matching the **title** and **author** in the database, and outputs the message **Title: title\tAuthor: author\tEdition: <E1(Q1), E2(Q2), ...>;**, where E1, E2, ... are sorted based on their editions, Q1, Q2, ... are the current numbers of borrowable copies, and the editions are wrapped in **< >**, if there is at least one record. Otherwise, output **Book doesn't exist.;**.

- **Find Author**

To find all books written by an author takes a string, **author**, as input, finds all books written by **author**, and outputs the message **author's Books: <T1, T2, ...>;**, where T1, T2, ... are sorted with string relation operator **<** and wrapped in **< >**, if there is at least one datum. Otherwise, output **No book found.;**.

- **Sort by Title**

To sort the database based on the **title** takes no input, sorts the database by comparing the **title** with string relation operator **<** while comparing the **author** with string operator **<** for those having the same **titles**, and outputs the result in the format **Title: title\tAuthor: author\tEdition: <E1(Q1), E2(Q2), ...>;** for each record, where E1, E2, ... are sorted based

on their editions, Q1, Q2, ... are the current numbers of borrowable copies, and the editions are wrapped in `< >`.

- **Sort by Author**

To sort the database based on the **author** takes no input, sorts the database by comparing the **author** with string relation operator `<` while comparing the **title** with string operator `<` for those having the same **authors**, and outputs the result in the format **Title: title\tAuthor: author\tEdition: <E1(Q1), E2(Q2), ...>**; for each record, where E1, E2, ... are sorted based on their editions, Q1, Q2, ... are the current numbers of borrowable copies, and the editions are wrapped in `< >`.

Borrowing Operations

The library database supports the following commands for book lending:

- **Borrow**

To borrow a book takes three strings, **title**, **author** and **borrower**, and an integer **edition** as input. If the specified **title**, **author**, and **edition** do not match any record in the database, output the message **Datum doesn't exist.;**. Otherwise, a **borrower** may hold at most one copy of the same edition at any time; if the same **borrower** has already borrowed this edition, output the message **Already borrowed.;**. Otherwise, if the edition exists but no copy is currently available, output the message **No available copies.;**. Otherwise, decrement the number of borrowable copies of that edition by 1 and output the message **borrower borrows author's title, Edition: edition.;**.

- **Return**

To return a book takes three strings, **title**, **author** and **borrower**, and an integer **edition** as input, locates the corresponding borrow record, increments the number of borrowable copies of that edition by 1, and outputs the message **borrower returns author's title, Edition: edition.;** on success. Otherwise, output the message **Borrow record not found.;**.

Commands

There are eight different commands while each command is issued in a line.

1. **Insert:** Insert "title" "author" edition quantity
2. **Delete Edition:** Delete Edition "title" "author" edition
3. **Find Book:** Find Book "title" "author"
4. **Find Author:** Find Author "author"
5. **Sort by Title:** Sort by Title
6. **Sort by Author:** Sort by Author
7. **Borrow:** Borrow "title" "author" edition "borrower"
8. **Return:** Return "title" "author" edition "borrower"

Note: All test cases guarantee that the input is well-formed (proper quoting and types). The system distinguishes commands by their exact spelling (case-sensitive), so any unrecognized command results in **Unknown Command.;** , and any command missing required arguments results in **Incomplete Command.;** .

The system will produce output as follows:

1. The output message with corresponding input command.
2. If the command doesn't exist, output the message **Unknown Command.;** .
3. If the command is not complete, output the message **Incomplete Command.;** .

See the sample output below for examples.

Input

1. Complete the required class implementation. The following files already contain some implementations:
 - `command.h`
2. The Online Judge system will replace the following files. **DON'T** write implementation in those file:
 - `entrypoint.cpp`
 - `test.h`
 - `case.h`
3. The test cases for student available in `case*.h`.
4. The Online Judge captures the commands and calls the `execute` method. You must implement this in `command.h`; the command argument contains each line from the `inputCommand` defined in `case*.h`.

Output

Online Judge will run the test:

1. When any test failed, Online Judge will try to list error messages.
2. The Online Judge will capture your output. Please refer to the `expected` variable in `case*.h`.